

velocoach-rs: System Design Document and 20-Day Implementation Plan

Note on process: I never completed the required enrichment pass. Every attempt to submit the draft for enrichment was rejected for exceeding a 50,000-character limit, and I ran out of turns before cutting it to fit. Vague or hedged claims were therefore not replaced with named-source specifics. This is the text as I last wrote it, shortened in places (the full Swift package snippet, the per-crate Cargo details, two resilience-matrix rows and a fuller ENGINEERING_NOTES outline were cut).

Your architecture holds up, and you can build it as planned with three changes:

1. Run cue decisions inside `ingest()`, not only at the 4 Hz `tick()`. A tick-only design adds up to 250 ms of decision delay.
2. Correct the W'_{bal} recovery constant to $W'/(CP - P)$. Plain W'/CP is only right when $P = 0$.
3. Report latency to audio submit (T_3) separately from the time sound reaches the ear (T_4). Bluetooth earbud (A2DP) output latency alone can use up most of a 150 ms budget.

Anything I could not check against a primary source is marked [UNVERIFIED].

TL;DR

- **The plan works with those three fixes.** Two latency targets: $T_0 \rightarrow T_3$ edge compute ($p_{99} < 150$ ms) and $T_0 \rightarrow T_4$ at the ear (reported, not promised).
- **Your GATT layouts are confirmed** against Bluetooth SIG XML characteristic definitions (the oesmith/gatt-xml mirror, plus a third-party copy for Sensor Location):
 - your CPM presence bits are correct;
 - HRM Energy Expended is uint16 in kJ;
 - CSC wheel data is uint32 revolutions + uint16 time at 1/1024 s.

I could not open the current GATT Specification Supplement PDF itself.

- **Build order:**

Days	Work
------	------

1	Radio smoke test
---	------------------

2-8	Pure-Rust crates and FFI
-----	--------------------------

9-11	Phones; tag v0.1.0 at end-to-end
------	----------------------------------

12-20	Latency, faults, golden logs, Intervals.icu, soak, docs
-------	---

A strict cut list protects the demo if you slip.

1. System Design Document

1a. Goals, non-goals, success criteria

Goals

- A hardware-in-the-loop demo over real BLE:
 - iQOO: a standard HR/CP/CSC sensor plus the custom VC-SIM service;
 - iPhone 12: the coach;
 - Mac: build machine and vc-cli harness.
- A deterministic sans-IO Rust core: the same inputs always give the same cue log.
- Measured latency with explicit clock-sync error bounds.
- Aki's rules as executable tests. From his article "Closed-Loop Cybernetic Intelligence in Cycling" (alphanome.ai):
 - a deterministic wrapper that bypasses the SLM when "HR > 98% HRmax for >120s"; (alphanome)
 - cues that are "Command verb first", have "Max 1 numerical parameter" and a "Maximum 8 to 12 total syllables"; (alphanome)
 - sensor rates of power 1-4 Hz and HR 1 Hz. (alphanome)

Non-goals: ML/SLM (the SafetyGate is the wrapper one would later sit behind), app-store distribution, ANT+, FTMS, Strava/Garmin (design note only), cloud.

#	Criterion	Target	Measured by
S1	Decision time (ingest → cue)	p99 < 1 ms on iPhone 12	<code>t2 - t1</code> , hdrhistogram
S2	T1→T3 (ingest → audio scheduled)	p99 < 20 ms	iOS stamps
S3	T0→T3 (sim send → audio submit)	p99 < 150 ms, ± rtt_min/2 shown	Synced stamps
S4	T0→T4 (adds <code>outputLatency</code>)	Report only	LATENCY.md
S5	Decoder robustness	0 panics; ≥ 10 min fuzz per target; proptest 10k cases	CI + local
S6	Determinism	Golden cue logs identical across runs and machines	<code>insta</code>
S7	Soak	60 min; 0 crashes; 0 unrecovered disconnects; < 0.5% TELEM seq loss	Ride log
S8	Phrase lint	100% verb-first, ≤ 1 number, 8-12 syllables	<code>cargo test</code>
S9	no_std	vc-gatt and vc-physio build for <code>thumbv7em-none-eabihf</code>	CI
S10	Hot path	0 heap allocations per <code>ingest()</code> after warm-up	Counting allocator

The S3/S4 split matters because the article targets a "sub-100-millisecond latency loop from threshold breach to auditory delivery", and earbud output latency is outside the code's control. `alphanome` Your ~160 ms A2DP figure is [UNVERIFIED]; measure it with `AVAudioSession.outputLatency`.

1b. Diagrams

Crates

- `vc-gatt` and `vc-physio` (both no_std) feed `vc-sim` and `vc-core`.
- `vc-sim` and `vc-core` feed `vc-ffi` (UniFFI), which generates the Swift and Kotlin bindings.
- `vc-sync` parses `.zwo` files into `vc-core::WorkoutPlan`.
- Tools: `vc-cli` (tokio + btleplug is fine here), `phrasegen`, `uniffi-bindgen`.

- The iQOO links `libvc_ffi.so` built with the `sim` feature; the iPhone links `libvc_ffi.a` built without it.

Connection setup

```

iQOO                                     iPhone
openGattServer; addService(HRS)→onServiceAdded ✓
addService(CPS)✓ → addService(CSCS)✓ → addService(VCSIM)✓
startAdvertising(adv, scanResp) — ADV_IND —————→
scanForPeripherals([1818]) → didDiscover
onConnectionStateChange(CONNECTED) ←— CONNECT — connect(p)
                                     ←— discovery —
discoverServices/Characteristics
onCharacteristicReadRequest ←— read 2A65/2A5D/2A5C
onDescriptorWriteRequest(2902 = 01 00) ←— CCCD — setNotifyValue(true,
HRM, CPM, CSC, TELEM)
sendResponse(GATT_SUCCESS) —————→
didUpdateNotificationState
notify(HRM) — HVN —————→ didUpdateValueFor → T1 →
ingest
onNotificationSent ✓ → send next queued packet

```

Steady state

- **Android:** the "vc-radio" HandlerThread calls `sim.step(now)` every 50 ms into a bounded queue (64 entries, drop-oldest per characteristic).
 - If nothing is in flight: stamp `t0`, notify, set `inFlight`.
 - `onNotificationSent` clears the flag and drains the next packet. A TELEM STAMP follows every sensor packet.
- **iOS:** `didUpdateValueFor` stamps `t1 = clock_gettime_nsec_np(CLOCK_UPTIME_RAW)`. `ingest` then runs `decode` → `physio` → `SafetyGate` → `Arbiter` → optional Cue (`t2`), and Swift plays it (`t3`).

Clock sync (Cristian/NTP)

```

iPhone a — CTRL PING{id,a} (write w/o response) —→ iQOO b = rx time
iPhone d ←— TELEM PONG{id,b,c} ————— c = time before notify
rtt = (d-a) - (c-b);  θ = ((b-a) + (c-d))/2;  keep min-rtt of last 16;  |ε| ≤
rtt_min/2
t0_ios = t0_android - θ

```

Cue to audio

```

ingest → SafetyGate → Candidate(Safety) ↵
      → Effort rules (W'bal hysteresis) |→ Arbiter.select(now) →
Cue{id,phrase,t2}
tick  → Structure (countdowns, steps)    | rejects: stale>2 s, cooldown,
gap<8 s, speaking
      → Info (decoupling) _____]
Swift: activate session (duck) → scheduleBuffer → t3 → on_audio_submitted(id,
t3, outputLatency)
      t4 = t3 + outputLatency + ioBufferDuration → completion →
on_audio_finished → deactivate

```

Reconnect

1. `didDisconnectPeripheral` → `engine.on_link(Down)`. Channels age to Stale (3 s), then Lost (10 s).
2. Call `connect(p)` again immediately; iOS keeps a pending connect with no timeout.
3. In parallel, run `scan([1818])` in case Android rotated its address.
4. On connect: rediscover, re-subscribe, send 8 PINGs to resync, then Recovering → 3 good samples → Fresh.
5. Android clears its CCCD map and in-flight flag, and restarts advertising if it stopped.

Fault injection

1. vc-cli or the iPhone debug sheet writes `CTRL_FAULT{Dropout, HR, 12000 ms}`.
2. Kotlin's `onCharacteristicWriteRequest` sends a response if one is needed, then calls `sim.apply_ctrl(direct ByteBuffer)`; Rust parses and validates.
3. HR goes silent for 12 s. The iPhone shows Stale → Lost → Recovering → Fresh, which the golden logs check.

1c. Byte-level specifications (all little-endian)

Heart Rate Measurement 0x2A37 (service 0x180D, notify; from the SIG XML)

Bit Meaning

0 0 = HR uint8 bpm; 1 = HR uint16 bpm

1-2 Sensor contact: 0/1 not supported; 2 supported, no contact; 3 contact detected

3 Energy Expended present: uint16 kJ. The XML unit tag says joule; the bit text and the HRS spec say kJ.

4 RR present: uint16[n] at 1/1024 s, oldest first; n comes from the remaining length ([github](#))

5- RFU (write 0, ignore on decode)

7

- The sim sends flags ([0x16](#)), uint8 HR and one RR value: 4 bytes.
- In a 20-byte payload, the maximum is 9 RR values (uint8 HR, no energy) or 8 (uint16 HR). ([mariam](#))
- The HRS spec gives the typical rate as "approximately 1 time per second". ([bluetooth](#))

Cycling Power Measurement 0x2A63 (service 0x1818): ([flags u16 | inst_power s16 W](#)) (mandatory), then optional fields in bit order: ([github](#))

Bit	Field	Type	Adds a field?
0	Pedal Power Balance	u8, 1/2 %	yes
1	Balance Reference (0 unknown, 1 left)	—	no
2	Accumulated Torque	u16, 1/32 N·m	yes
3	Torque Source (0 wheel, 1 crank)	—	no
4	Wheel Rev	u32 revs + u16 time at 1/2048 s	yes
5	Crank Rev	u16 revs + u16 time at 1/1024 s	yes
6	Extreme Force	4 bytes	yes
7	Extreme Torque	4 bytes	yes
8	Extreme Angles	3 bytes	yes
9 / 10	Top / Bottom Dead Spot	u16 degrees each	yes
11	Accumulated Energy	u16 kJ	yes
12	Offset Compensation Indicator	—	no

- The sim sends flags `0x0020` (crank only): 8 bytes. With balance and energy, `0x0821`: 11 bytes.
- Decoders reject truncated packets and never panic.
- The internal types of bits 6-7 are [UNVERIFIED], but only their lengths matter, because the sim never encodes them.

CP Feature 0x2A65 (read, u32) [github](#)

- Bits 0-11: Pedal Power Balance, Accumulated Torque, Wheel Rev, Crank Rev, Extreme Magnitudes, Extreme Angles, Top/Bottom Dead Spot, Accumulated Energy, Offset Compensation Indicator, Offset Compensation, CPM Content Masking, Multiple Sensor Locations. [github](#) [github](#)
- The exact indices of bits 12 and up are [UNVERIFIED].
- The sim sends `0x00000089` (balance + crank + energy).
- The Engine trusts this Feature read, not whatever the first notification happens to contain. [github](#)

Sensor Location 0x2A5D (read, u8)

- 0 Other, 1 Top of shoe, 2 In shoe, 3 Hip, 4 Front Wheel, 5 Left Crank, 6 Right Crank, 7 Left Pedal, 8 Right Pedal, 9 Front Hub, 10 Rear Dropout, 11 Chainstay, 12 Rear Wheel, 13 Rear Hub, 14 Chest, 15 Spider, 16 Chain Ring, 17-255 RFU. [github](#)
- These come from a third-party copy of the SIG XML.
- The sim reports 5 (Left Crank) for CPS and 6 (Right Crank) for CSCS.

CSC Measurement 0x2A5B (service 0x1816)

- `flags u8` (bit 0 wheel, bit 1 crank), then optional `[u32 wheel revs + u16 time at 1/1024 s]`, then optional `[u16 crank revs + u16 time at 1/1024 s]`. [github](#) [bluetooth](#)
- CSC wheel time is 1/1024 s but CPM wheel time is 1/2048 s. Mixing them up is a classic bug.
- The sim sends crank only: 5 bytes.

CSC Feature 0x2A5C (u16)

- Bit 0 wheel supported, bit 1 crank supported, bit 2 multiple locations. The sim sends `0x0002`. [github](#) [bluetooth](#)
- Whether SC Control Point 0x2A55 is required in this case is [UNVERIFIED]. Advertising crank-only avoids the question.

VC-SIM custom service

- Generate your own UUIDs once with `uuidgen` and freeze them. Example:
 - service `6E7C0001-8A3B-4F2E-9D5C-1B2A3C4D5E6F`

- TELEM `(...0002...)`: notify + CCCD
- CTRL `(...0003...)`: write + write-without-response
- The default ATT MTU of 23 leaves a 20-byte payload.
- Every packet starts with `(ver u8 = 1 | op u8)`. Decoders reject any other version.

Dir	Op	Name	Body	Bytes
TELEM	0x01	STAMP	seq u16, char_id u8 (1 HRM, 2 CPM, 3 CSC), char_seq u16, t0_ns u64	15
TELEM	0x02	PONG	ping_id u16, t_rx u64, t_tx u64	20
TELEM	0x03	STATUS	seq u16, scenario u8, state u8, faults u16, q_drops u16, notify_errs u16	12
CTRL	0x10	PING	ping_id u16, t_client u64	12
CTRL	0x20	LOAD	scenario u8, seed u32	7
CTRL	0x21/0x22	START/STOP	—	2
CTRL	0x30	FAULT	kind u8, ch u8, dur_ms u32, param i16	10
CTRL	0x31	CLEAR	kind u8 (0xFF = all)	3
CTRL	0x40	RATE	ch u8, hz_x10 u8	4

Fault kinds: 1 Dropout, 2 Garbage, 3 Disconnect, 4 Coast, 5 HrRedline, 6 WprimeMismatch (`(param)` = true W' in units of 100 J).

Advertising budget (legacy advertising, 31 bytes per packet)

- Advertisement: Flags (3) + 16-bit UUID list 180D/1818/1816 (8) = 11 of 31 bytes.
- Scan response: 128-bit VC-SIM UUID (18) + name "VCSIM" (7) = 25 of 31 bytes.
- iOS filters on 1818, which is in the primary advertisement.

1d. Rust data model and APIs

```
rust
```

```

// vc-gatt (no_std)
pub enum ChannelId { Hr = 1, Power = 2, Csc = 3 }
pub struct Hrm { pub bpm: u16, pub contact: Contact, pub energy_kj: Option<u16> }
pub enum Contact { NotSupported, NoContact, Contact }
pub struct Cpm { pub flags: u16, pub power_w: i16, pub balance_half_pct: Option<u16>,
    pub wheel: Option<(u32,u16)>, pub crank: Option<(u16,u16)>, pub acc_energy_kj: Option<u16> }
pub struct Csc { pub wheel: Option<(u32,u16)>, pub crank: Option<(u16,u16)> }
pub struct CpFeature(pub u32); pub struct CscFeature(pub u16); pub struct Sensor { pub id: u8, pub val: u16 }
pub enum DecodeError { Empty, Truncated { need: u8, have: u8 }, BadVersion(u8),
    pub fn decode_hrm(b: &[u8]) -> Result<Hrm, DecodeError>; // likewise cpm/csc
pub fn encode_hrm(m: &Hrm, out: &mut [u8; 20]) -> usize; // no allocation
pub enum Telem { Stamp { seq: u16, char_id: u8, char_seq: u16, t0_ns: u64 }, Power { power_w: i16, energy_kj: Option<u16> },
pub enum Ctrl { Ping { id: u16, t: u64 }, Load { scenario: u8, seed: u32 }, Status { status: u8, fault: Fault },
    Fault { kind: FaultKind, ch: u8, dur_ms: u32, param: i16 }, Clear(u8), Rate { rate: f64 } }
// vc-physio (no_std + libm)
pub struct Sample { pub t_ns: u64, pub ch: ChannelId, pub value: f64 } // W,
pub enum Quality { Fresh, Stale, Lost, Recovering { good: u8 } }
pub enum WbalModel { Differential(Integration), WaterworthIntegral { tau_s: f64 } }
pub enum Integration { Exact, GcEuler1Hz }
pub struct WbalState { pub cp_w: f64, pub wprime_j: f64, pub w_j: f64, last_t_ns: u64 }
pub struct CadenceDeriver { prev: Option<(u16,u16)>, last_change_ns: u64, pub rpm: i16 }
// vc-sim: Truth { t_ns, power_w, hr_bpm, cadence_rpm, wprime_true_j, blown },
// Fault { Dropout, Garbage, Disconnect, Coast(Stale|Silent), HrRedline, WprimeTrueJ, WprimeJ, WprimeJErr }
// vc-core
pub struct WorkoutPlan { pub name: String, pub steps: Vec<Step> }
pub struct Step { pub start_s: u32, pub dur_s: u32, pub kind: StepKind, pub target: Target }
pub enum StepKind { Warmup, Interval, Recovery, Cooldown, Steady }
pub enum Target { PowerW { lo: u16, hi: u16 }, Free }
pub enum Priority { Info = 0, Effort = 1, Structure = 2, Safety = 3 }
pub enum CueKind { HrRedline, SensorLost(ChannelId), SensorBack, LinkDown, LinkUp, WbalLow { pct: u8 }, WbalRecovered, PowerHigh, PowerLow, CadenceLow, Coasting }
pub struct Candidate { pub kind: CueKind, pub prio: Priority, pub created_ns: u64 }
pub struct Cue { pub id: u64, pub kind: CueKind, pub prio: Priority, pub phrase: String }
pub struct LatencyRecord { pub cue_id: u64, pub t0_ns: Option<u64>, pub t1_ns: u64 }
pub struct EngineConfig { pub cp_w: f64 /*250*/, pub wprime_j: f64 /*20000*/, pub redline_frac: f64 /*0.98*/, pub redline_hold_s: f64 /*120*/, pub wbal_fir: f64 /*1000*/,
    pub global_gap_s: f64 /*8*/, pub stale_drop_s: f64 /*2*/, pub stale_s: f64 /*1000*/, pub decouple_warmup_s: f64 /*600*/, pub decouple_min_s: f64 /*1200*/, pub decouple_max_s: f64 /*1200*/ }
pub enum EngineError { Decode(DecodeError), NonMonotonicTime { prev: u64, now: u64 } }
// vc-core::Engine mirrors the FFI methods below with &mut self and core types.
// vc-sync
pub fn parse_zwo(xml: &str) -> Result<WorkoutPlan, ZwoError>; // SteadyState,
pub fn events_path(oldest: &str, newest: &str) -> String; // /api/v1/athl
pub fn parse_events(json: &str) -> Result<Vec<PlannedWorkout>, SyncError>; //

```

UniFFI surface (uniffi =0.32.2, proc macros)

rust

```
uniffi::setup_scaffolding!();
#[derive(uniffi::Enum, Clone, Copy)] pub enum Source { Hr, Power, Csc, Telem }
#[derive(uniffi::Enum, Clone, Copy)] pub enum CuePriority { Info, Effort, Struc
#[derive(uniffi::Record)] pub struct FfiConfig { pub cp_w: f64, pub wprime_j: f
    #[uniffi(default = 0.98)] pub redline_frac: f64, #[uniffi(default = 8.0)] pub
#[derive(uniffi::Record)] pub struct CueOut { pub id: u64, pub phrase_id: u16,
#[derive(uniffi::Record)] pub struct Snapshot { pub power_w: f64, pub hr_bpm: f
    pub decoupling_pct: Option<f64>, pub hr_q: String, pub pw_q: String, pub step
#[derive(Debug, thiserror::Error, uniffi::Error)]
pub enum VcError { #[error("decode: {reason}")] Decode { reason: String }, #[er
    #[error("config: {reason}")] Config { reason: String } }
#[derive(uniffi::Object)] pub struct Engine { inner: std::sync::Mutex<vc_core::
#[uniffi::export]
impl Engine {
    #[uniffi::constructor] pub fn new(cfg: FfiConfig, zwo: Option<String>) -> Res
    pub fn ingest(&self, t_ns: u64, source: Source, bytes: &[u8]) -> Result<Optio
    pub fn tick(&self, t_ns: u64) -> Option<CueOut> { todo!() }
    pub fn on_link(&self, up: bool, t_ns: u64) { todo!() }
    pub fn on_audio_submitted(&self, cue_id: u64, t3_ns: u64, output_latency_ns:
    pub fn on_audio_finished(&self, cue_id: u64, t_ns: u64) { todo!() }
    pub fn make_ping(&self, t_ns: u64) -> Vec<u8> { todo!() }
    pub fn snapshot(&self) -> Snapshot { todo!() }
    pub fn latency_csv(&self) -> String { todo!() }
}
#[cfg(feature = "sim")] #[derive(uniffi::Record)] pub struct SimPacket { pub ch
#[cfg(feature = "sim")] #[derive(uniffi::Object)] pub struct Simulator { inner:
#[cfg(feature = "sim")] #[uniffi::export]
impl Simulator {
    #[uniffi::constructor] pub fn new(scenario_toml: String, seed: u64) -> Result
    pub fn step(&self, now_ns: u64) -> Vec<SimPacket> { todo!() }
    pub fn apply_ctrl(&self, now_ns: u64, bytes: &[u8]) -> Result<Option<Vec<u8>>
}
```

UniFFI 0.32 notes:

- `&[u8]` arguments are zero-copy. Swift passes `Data`. [github](#)
- Kotlin must pass a **direct** `ByteBuffer`:
`ByteBuffer.allocateDirect(n).put(arr).flip()`. [github](#) [github](#)
- Borrowed `&[u8]` is rejected on `async fn`, so keep the API synchronous. [github](#)
- Returned `Vec<u8>` still copies. That is fine on cold paths.

1e. Algorithms

Cadence (handles u16 wrap)

```
if prev None: store, return
drev = revs.wrapping_sub(prev.revs); dt = t.wrapping_sub(prev.t)
if drev==0 && dt==0: if now-last_change > 2.5 s → rpm = 0 (coasting/stale);
return
if drev==0 || dt==0: garbage, return
if now-last_change > 60 s: rebaseline (event time wraps every 64 s)
rpm = 60·drev·1024/dt; if rpm > 250 → garbage; store; last_change = now
```

Speed is $(drev \cdot circumference / (dt / 2048))$ for CPM and $(/1024)$ for CSC.

W'bal, differential model (GoldenCheetah's default)

GoldenCheetah's WPrime.cpp resamples power to 1 s, fills gaps with zeros, and steps:

[github](#)

```
if (smoothedValue < CP) W = W + (CP-smoothedValue)*(WPRIME-W)/WPRIME;
else
    W = W + (CP-smoothedValue);
```

- The recovery constant is therefore $W' / (CP - P)$. Mark Liversedge's blog phrase "implicit $\tau = W' / CP$ " is the $P = 0$ case. [blogspot](#)
- GC's defaults are $CP = 250$ W and $W' = 20000$ J. [github](#)

For irregular BLE timing, use the exact solution over each interval:

```
dt = (t - last)/1e9; if dt ≤ 0 → ignore; gap > 10 s → P = 0 over gap (GC
zero-fill)
P ≥ CP: W -= (P-CP)·dt      else: W = W' - (W'-W)·exp(-(CP-P)·dt/W')
```

- Keep a [GcEuler1Hz](#) mode for parity tests against GoldenCheetah.
- Clamp $W \leq W'$. Allow negative values internally, as GC does, but floor the UI at 0%. [github](#)

W'bal, Waterworth/Skiba integral model (optional)

- GC computes $\tau = 546 \cdot \exp(-0.01 \cdot (CP - \text{mean power below } CP)) + 316$. [github](#)
- Integrator: $I += \exp(t/\tau) \cdot \max(P-CP, 0); \text{ out} = \exp(-t/\tau) \cdot I$. [github](#)
- For live use without overflow, use the recurrence $S = S \cdot \exp(-dt/\tau) + \max(P-CP, 0) \cdot dt$.

Test vectors (CP 250, W' 20000)

Input	Expected W
350 W for 100 s	10000 J
then 100 W for 133.33 s	≈ 16321 J
$P = CP$	constant
$W = 0$, then $P = 0$ for 80 s	≈ 12642 J

Aerobic decoupling

Definitions:

- TrainingPeaks' Aerobic Decoupling "compares the Efficiency factor from the first half of an activity to the second half". It treats under 5% as sound aerobic endurance and warns that values "under 20 minutes in duration aren't as valid". (trainingpeaks) (evokeendurance)
- Runalyze: $Pw:Hr [\%] = (AE_1 - AE_2) / AE_1$. (runalyze)

Live method:

1. Use 30 s bins.
2. Skip the first 600 s, bins with $P < 0.4 \cdot CP$, and bins whose quality is not Fresh.
3. Steady-state gate: power coefficient of variation < 0.10 over the last 10 bins.
4. Once at least 1200 s of valid bins exist, split the most recent valid window (up to 60 min) into halves. $EF = \Sigma P / \Sigma HR$ for each half; $dec = 100 \cdot (EF1 - EF2) / EF1$.
5. Fire an Info cue at $\geq 5\%$; re-arm below 3.5%.

Quality state machine

- Fresh $\rightarrow$ Stale when age > 3 s; Stale $\rightarrow$ Lost when age > 10 s.
- A good sample moves Stale or Lost to Recovering. The 3rd consecutive good sample moves it to Fresh.
- Garbage packets never count as good.

SafetyGate

- **HrRedline:** HR is Fresh and $> 0.98 \cdot HR_{max}$ continuously for ≥ 120 s. It re-arms after $HR < 0.95 \cdot HR_{max}$ for 30 s. Cooldown 60 s.
- **SensorLost:** fires once per transition to Lost.
- Safety cues bypass cooldowns and the global gap, and are the **only** cues allowed to talk over a playing cue.

Arbiter (no allocation)

```

slots: [Option<Candidate>; 16], one per kind (newest wins)
drop if now - created > 2 s; drop if in per-kind cooldown (WbalLow 90 s,
Power 45 s, Cadence 60 s, Drift 600 s)
non-Safety waits while speaking, or if now - last_end < 8 s, or if a
Structure cue is due within 8 s (reservation)
pick max priority, tie → oldest; set cooldown; speaking_until = now + dur_ms
WbalLow: armed ∧ wbal < 25% → fire, disarm; ¬armed ∧ wbal > 35% → arm
Steps: Countdown{10} at boundary - 10 s ("Get ready, interval in ten
seconds"); StepStart at boundary

```

Rider simulator

All constants below are tunable defaults, not cited physiology [UNVERIFIED].

- **HR target:** $HR_{ss} = HR_{rest} + (0.92 \cdot HR_{max} - HR_{rest}) \cdot P_{eff}/CP$.
- **HR dynamics:** $dHR/dt = (HR_{ss}(P_{eff}(t - 15\text{ s})) - HR)/\tau$, with $\tau_{on} = 35\text{ s}$ rising and $\tau_{off} = 60\text{ s}$ falling.
- **Cardiac drift:** $P_{eff} = P \cdot (1 + \alpha \cdot E_{kJ}/1000)$, with α calibrated so 60 min at $0.65 \cdot CP$ gives about 5% decoupling.
- **True W':** tracked with the exact model. At 0 the rider "blows up" and power is capped at $\min(\text{target}, 0.92 \cdot CP)$ until $W > 0.1 \cdot W'$.
- **Noise:** power $N(0, (0.03P)^2)$ and HR $\pm 1\text{ bpm}$, using a seeded `Pcg64Mcg` so runs are reproducible.
- **Crank events:** u16 wrapping revolution counter; event time = $((t_s \cdot 1024) \text{ as } u16)$. `teaandtectime`
- **Coast modes:** Stale repeats the last (revs, time) pair; Silent sends no CSC packets.

Clock sync parameters

- Keep the minimum-RTT sample of the last 16. Resync every 30 s and after every reconnect.
- My estimate: $rtt_{min} \approx 20\text{--}40\text{ ms}$, giving an error bound of about $\pm 10\text{--}20\text{ ms}$ [UNVERIFIED].

Latency percentiles

- Crate `hdrhistogram` 7.6.0: `Histogram::<u64>::new_with_bounds(1, 10_000_000_000, 3)`, recording nanoseconds. `docs`
- Record with `record(v)` (returns a `Result`) or `saturating_record`; read with `value_at_quantile(0.99)`. `docs`
- One histogram per latency segment, allocated in `Engine::new`.

1f. Threading

- **Rust:** a synchronous state machine behind a `Mutex` per UniFFI Object (UniFFI requires `Send + Sync`). `github` The lock is uncontended because each platform calls from one serial context.
- **Why not tokio on the phones:**
 - The OS already delivers radio events (CoreBluetooth's queue, Android's binder callbacks).
 - An async runtime would add threads, wake-up jitter, binary size and a second scheduler.
 - A sans-IO core replays bit-identically on the Mac.
 - Tokio stays in vc-cli only.
- **iOS:**
 - `CBCentralManager(delegate:, queue: bleQueue)` with `.userInteractive` QoS. All Engine calls happen on `bleQueue`.
 - A `DispatchSourceTimer` calls `tick()` every 250 ms.
 - `snapshot()` is published to an `@Observable` model on the main actor.
 - `AudioCuePlayer` runs on its own serial queue, because `AVAudioSession.setActive` can block.
- **Android:** binder callbacks immediately `post{}` to `HandlerThread("vc-radio")`. That thread owns the Simulator, the notify queue and the in-flight flag.

1g. Resilience matrix

Failure	Detection	Handling	User sees	Test
Malformed packet	<code>DecodeError</code>	Count it; don't refresh quality; never panic	Debug counter	Fuzz; Garbage golden log
Stale / lost channel	Quality state machine	Drop Effort cues that depend on it; Safety cue on Lost	"Check your sensor..."	Dropout golden log
Disconnect	iOS / Android callbacks	Reconnect + rescan, resync clock, re-subscribe	Status pill	Disconnect fault
Notify failure	Non-SUCCESS return or callback status	Retry once after 20 ms, then drop and count	STATUS notify_errs	Soak
Adapter off	<code>centralManagerDidUpdateState</code> / <code>ACTION_STATE_CHANGED</code>	Pause, then resume	Banner	Toggle Bluetooth
Permission denied	Authorization APIs	Explanation screen + Settings link	Blocking screen	Manual
iOS GATT cache	Characteristic missing after a table change	Freeze the table; toggle Bluetooth / forget device	Error text	Change table once
Vivo background kill	No STATUS for > 5 s	Foreground service + battery "unrestricted" + screen on [UNVERIFIED menu names]	Disconnect flow	60 min screen-off
Audio interruption / route change	<code>AVAudioSession</code> notifications	Mute on begin; restart on end + shouldResume; drop cues > 2 s old	Cue skipped	Pull earbuds

1h. Cue phrase bank

Lint test

- The first word must be in `(VERBS)`.
- At most one number phrase; a run of number words counts as one.
- 8-12 syllables, counted from a checked-in `(phrases/syllables.tsv)`. Unknown words fail the test.

Aki's own example, "Cadence up. Target ninety.", is 7 syllables and starts with a noun, so it fails his own rule; E09 rewrites it to 8 syllables, verb first. `(alphanome)`

Safety

ID	Phrase	Syll.
S01	Stop pushing now, ease off and breathe	8
S02	Ease off now, your heart rate is too high	9
S03	Drop the power, heart rate near max	8
S04	Back off now, spin easy for a minute	10
S05	Check your sensor, heart rate signal lost	9
S06	Check your power meter, signal lost	9
S07	Keep riding, sensors are back online	9
S08	Pause the workout, sensors reconnecting	10
S09	Resume the workout, all sensors back	9

Structure

ID	Phrase	Syll.
T01	Get ready, interval in {ten, thirty} seconds	10/11
T02	Start the interval now, hold {N} watts	11
T03	Begin recovery, spin easy now	10
T04	Start your cool down, spin it out easy	9
T05	Finish strong, {ten, thirty} seconds to go	8/9
T06	Begin the warm up, keep it easy	9
T07	Hold steady, last interval coming up	10
T08	Recover now, next effort in {ten, thirty} seconds	11/12
T09	Finish up, the workout is complete	9
T10	Hold {N} watts for this whole block	9

Effort

ID	Phrase	Syll.
E01	Ease off, your reserve is at {ten, twenty, thirty} percent	10/11
E02	Save your matches, reserve is running low	10
E03	Sit in and recover your reserve	9
E04	Lower the power to {N} watts	10
E05	Raise the power to {N} watts	9
E06	Hold this power, you are on target	9
E07	Push a little harder, you have room	9
E08	Back off a bit, you are over target	10
E09	Raise your cadence, target {ninety, eighty five}	8/9
E10	Lower your cadence, settle at ninety	10
E11	Keep pedaling, don't stop the cranks	8
E12	Spin faster, lighten the gear now	8
E13	Shift to an easier gear and spin	9
E14	Note your reserve is back to full	8
E15	Go now, you have energy to spare	9
E16	Wait for the climb, then go harder	8

Info

ID	Phrase	Syll.
I01	Ease the pace, your heart rate is drifting	9
I02	Drink some water, heart rate is drifting up	10
I03	Eat something small, then keep riding	8
I04	Stay smooth, decoupling at {five, ten} percent	9
I05	Keep it steady, heart and power are matched	10
I06	Relax your shoulders and breathe deep	8
I07	Stay aero, keep your effort even	9
X01	Test audio, coach voice is working	9

Number vocabulary

- $\{N\} \in$ two hundred, two twenty, two forty, two fifty, two sixty, two eighty, three hundred, three twenty. Each is 3 syllables; targets snap to the nearest value.
- Also needed: five, ten, twenty, thirty, ninety, eighty five.
- That gives about 70 WAV files in total.

Files and manifest

- Name files `{ID}_{slot}.wav`.
- `manifest.json` entries look like:
`{"id":17,"key":"E01_twenty","kind":"WbaLLow","text":"...","syllables":11,"file":"E01_twenty.wav","dur_ms":2140,"sha256":"..."}`.
- A generated `phrase_table.rs` gives the core each phrase's `dur_ms`, which it needs for `speaking_until`.

Generation: `echo "$TEXT" | piper --model en_US-ljspeech-medium.onnx --output_file out.wav`, then `ffmpeg (silenceremove) + (loudnorm)`. The Piper CLI flags and the voice license are [UNVERIFIED].

1i. Platform specifics

Android manifest

xml

```

<uses-feature android:name="android.hardware.bluetooth_le" android:required="true"/>
<uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE"/>
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_CONNECTED_DEVICE"/>
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
<service android:name=".SensorRigService" android:exported="false"
    android:foregroundServiceType="connectedDevice"/>

```

Android rules and setup

- **Android 14+ foreground services:** you "must declare an appropriate service type" plus the matching permission. Otherwise `startForeground()` throws `MissingForegroundServiceTypeException`. ([android](#)) ([android](#))
 - `connectedDevice` also needs one runtime prerequisite, such as a granted `BLUETOOTH_ADVERTISE` or `BLUETOOTH_CONNECT`. ([github](#))
 - `ADVERTISE`, `CONNECT` and `SCAN` are runtime permissions. The peripheral doesn't need `SCAN`. ([android](#))
- **Startup flow:**
 1. Request `[BLUETOOTH_ADVERTISE, BLUETOOTH_CONNECT, POST_NOTIFICATIONS]`.
 2. Start the service from the foreground Activity.
 3. Call `startForeground(id, n, FOREGROUND_SERVICE_TYPE_CONNECTED_DEVICE)`.
- **GATT setup:**
 - `addService` one service at a time, waiting for `onServiceAdded(GATT_SUCCESS)` each time, in the order `HRS` → `CPS` → `CSCS` → `VCSIM`. Only then start advertising.
 - Measurement characteristics and `TELEM: NOTIFY + CCCD 0x2902`.
 - Feature and Location: `READ`.
 - `CTRL: WRITE | WRITE_NO_RESPONSE`.
- **CCCDs:** keep a per-device map. Call `sendResponse` when `responseNeeded`. Only notify when the stored value is `01 00`. ([iut-fbleau](#)) ([unt](#))
- **Notifications:**
 - Use the API 33+ `notifyCharacteristicChanged(device, ch, false, value): Int`. The old overload is deprecated as "not memory safe". ([android](#))
 - The docs say: "When multiple notifications are to be sent, an application must wait for this callback to be received before sending additional notifications." ([android](#)) ([microsoft](#))

- **Advertising:** `ADVERTISE_MODE_LOW_LATENCY`, `ADVERTISE_TX_POWER_MEDIUM`, connectable, timeout 0, `setIncludeDeviceName(false)` in the advertisement itself.

iOS

- **Info.plist:**
 - `NSBluetoothAlwaysUsageDescription` (required since iOS 13; missing it crashes the app). `medium`
 - `UIBackgroundModes = [bluetooth-central, audio]`. `apple`
 - Use `CBCentralManagerOptionRestoreIdentifierKey`. `github`
- **Audio session:** Apple DTS's recipe for navigation-style voice prompts: category `.playback`, mode `.voicePrompt`, options `[.duckOthers, .interruptSpokenAudioAndMixWithOthers]`. `apple`
 - Ducking lasts from activation to deactivation, so activate per cue or per interval. `blogspot`
 - Deactivate with `.notifyOthersOnDeactivation`, off the main thread. `github` `blogspot`
- **AVAudioEngine:** one `AVAudioPlayerNode`; preload all phrases as `AVAudioPCMBuffer`s (~12 MB). Safety cues use `.interrupts` to cut off whatever is playing.
- **Signing (free account):** Personal Team; Developer Mode on; profiles expire every 7 days, so re-sign on days 14 and 20.
 - Whether a free team can enable Background Modes is [UNVERIFIED]. Test it on day 1.
- **Packaging:** local Swift package `apps/ios/VcCore` with `.binaryTarget(name: "VcFFI", path: "VcFFI.xcframework")`. The modulemap must declare `vc_ffiFFI`.

1j. Build system

Workspace `Cargo.toml`

toml

```

[workspace]
resolver = "2"
members = ["crates/*", "tools/vc-cli", "tools/phrasegen", "uniffi-bindgen"]
[workspace.dependencies]
uniffi = "=0.32.2"
libm = "0.2"
thiserror = "2"
proptest = "1"
insta = { version = "1", features = ["yaml"] }
rand_pcg = "0.3"
rand_distr = "0.4"
hdrhistogram = "7.6"
[profile.dev]
debug = "line-tables-only"
[profile.release]
lto = "thin"
codegen-units = 4

```

Crate-specific settings:

- Keep the default `panic = "unwind"` so UniFFI can turn panics into errors.
- **vc-ffi**: `crate-type = ["lib", "staticlib", "cdylib"]`, `name = "vc_ffi"`, and feature `sim = ["dep:vc-sim"]`.
- **uniffi-bindgen binary**: depends on `uniffi` with features `["cli"]`; `main` is just `uniffi::uniffi_bindgen_main()`. [github](#)
- **uniffi.toml**:
 - Kotlin: `package_name = "ai.velocoach.ffi"`, `cdylib_name = "vc_ffi"`.
 - Swift: `module_name = "vc_ffi"`, `ffi_module_name = "vc_ffiFFI"`.
 - In 0.32, `--config` "now expects a global config file", so rely on the crate-local file. [github](#)

`scripts/build-ios.sh`

```
bash
```

```
#!/usr/bin/env bash
set -euo pipefail
rustup target add aarch64-apple-ios aarch64-apple-ios-sim
T=target/nosim
cargo build -p vc-ffi --release --target aarch64-apple-ios --target-dir $T
cargo build -p vc-ffi --release --target aarch64-apple-ios-sim --target-dir $T
cargo run -q -p uniffi-bindgen -- generate --library $T/aarch64-apple-ios/release
rm -rf build/headers && mkdir -p build/headers
cp build/swift/vc_ffiFFI.h build/headers/
cp build/swift/vc_ffiFFI.modulemap build/headers/module.modulemap
cp build/swift/vc_ffi.swift apps/ios/VcCore/Sources/VcCore/
rm -rf apps/ios/VcCore/VcFFI.xcframework
xcodebuild -create-xcframework \
    -library $T/aarch64-apple-ios/release/libvc_ffi.a -headers build/headers \
    -library $T/aarch64-apple-ios-sim/release/libvc_ffi.a -headers build/headers
    -output apps/ios/VcCore/VcFFI.xcframework
```

Known issue: UniFFI $\geq$ 0.29 modulemaps add `use "Darwin"` and `use "_Builtin_*` lines. [uniffi-rs#2917](#) reports these break XCFramework consumers on Xcode 26/27 for the Simulator. [github](#) If you hit it, strip them with `sed -i '' '/^ *use "/d'` `build/headers/module.modulemap`.

`scripts/build-android.sh`

```
bash

#!/usr/bin/env bash
set -euo pipefail
rustup target add aarch64-linux-android
cargo install cargo-ndk --locked
export ANDROID_NDK_HOME="$HOME/Library/Android/sdk/ndk/<r30 dir>" # [UNVERIFIED]
cargo ndk -t arm64-v8a -P 26 -o apps/android/app/src/main/jniLibs build -p vc-ffi
cargo run -q -p uniffi-bindgen -- generate --library target/aarch64-linux-android
--language kotlin --out-dir apps/android/app/src/main/java
```

Gradle: compileSdk 35, minSdk 26, `abiFilters += "arm64-v8a"`, dependency `net.java.dev.jna:jna:5.17.0@aar` (needs $\geq$ 5.12; check for the latest).

Keeping the 8 GB Mac responsive

- `org.gradle.jvmargs=-Xmx2g`
- `CARGO_BUILD_JOBS=4`
- Never run Xcode and Android Studio builds at the same time.
- Fuzz with `-jobs=1`.

CI

- **rust** job on macos-14, in order:
 1. `cargo fmt --check`
 2. `cargo clippy --workspace --all-targets --all-features -- -D warnings`
 3. `cargo nextest run --workspace --all-features`
 4. `cargo build -p vc-gatt -p vc-physio --target thumbv7em-none-eabihf`
 5. `cargo test -p vc-core --test golden`
- **fuzz** job on Ubuntu nightly: `cargo fuzz run $t -- -max_total_time=60` for each of hrm, cpm, csc, telem, ctrl.

1k. Testing strategy

- **vc-gatt:**
 - Spec vectors for every flag combination, including truncation at every byte.
 - proptest round-trips (`decode(encode(x)) == x`) and never-panic on arbitrary input.
 - A cargo-fuzz target per decoder.
- **vc-physio:**
 - The analytic W'bal vectors above.
 - Exact vs GC-Euler mode agree within 0.5% over 1 h.
 - Property: $W \leq W'$.
 - The quality state machine's transition table.
 - A synthetic 5% drift is measured as $5 \pm 0.3\%$.
 - Cadence wraps correctly at $65535 \rightarrow 0$.
- **GoldenCheetah cross-check:**
 1. Export the sim's ground truth as 1 Hz CSV.
 2. Import it into GC with $CP/W' = 250/20000$ and the differential model.
 3. Compare GC's "Minimum W' bal" to `vc-cli wbal --gc-parity`. [github](#)
 4. Accept a $\pm 1.5\%$ difference (GC smooths power with a spline first). [github](#)
- **Loopback** (vc-sim $\rightarrow$ encode $\rightarrow$ ingest, no radio): decoded values equal ground truth.
- **Golden logs:** `insta` snapshots of the cue log for 8 scenarios.
- **Arbiter:** one table-driven test per rule.
- **Allocations:** a counting allocator asserts 0 allocations over 10k `ingest()` calls after warm-up.
- **Benchmarks:** criterion benches for the hot paths.
- **Latency protocol:**
 1. Run the `vo2_5x3` scenario three times with earbuds.

2. Run it once with the phone speaker as a control.

11. Observability

- **Core:** no logging in the hot path. Events go to a 1024-entry ring buffer that the platforms drain at 4 Hz. vc-cli uses `tracing`.
- **Platform logs:** iOS `Logger(subsystem: "dev.zakir.velocoach")`; Android `Log.d("VC/radio")`.
- **Ride log (JSONL)**, one object per event:
 - `{"t":"sample","t_ns":...,"ch":"pw","v":312,"q":"Fresh"}`
 - `{"t":"cue","id":7,"key":"E01_twenty","t1":...,"t2":...,"t3":...,"t4":...}`
 - `sync`, `fault` and `link` events in the same format.
- **Latency CSV columns:**
`cue_id,t0_ns,t1_ns,t2_ns,t3_ns,t4_ns,sync_err_ns,route`.
- **DEVLOG.md** each day: Done / Measured / Broke / Decision / Tomorrow.

2. Implementation Plan (20 days × 7 h)

"DoD" = definition of done for the day.

Day 1: radio smoke test

Hour	Task
H1	Install toolchains
H2	nRF Connect GATT server on the iQOO; LightBlue on the iPhone
H3-H4	<code>apps/ios/Scanner</code> : scan, connect, subscribe, print hex
H5	Workspace + CI
H6	Push
H7	Fallback: test with btleplug on the Mac

DoD: HRM bytes from the iQOO show up on the iPhone.

Day 2: vc-gatt HRM/CSC

Hour	Task
H1	<code>cursor.rs</code> , <code>error.rs</code>
H2-H3	<code>hrm.rs</code> + 12 test vectors
H4	<code>csc.rs</code>
H5	proptest
H6	Sensor Location
H7	Clippy

DoD: vc-gatt tests green.

Day 3: CPM, VC-SIM, fuzz

Hour	Task
H1-H2	<code>cpm.rs</code>
H3	<code>cp_feature.rs</code>
H4	<code>vcsim.rs</code>
H5	proptest
H6	Fuzz targets, 10 min each
H7	no_std CI job

DoD: S5 and S9 met.

Day 4: vc-physio

Hour	Task
H1	W'bal, exact mode
H2	GC-Euler mode
H3	Waterworth model
H4	Analytic tests
H5	Quality state machine
H6	Cadence deriver
H7	DEVLOG

Day 5: decoupling + simulator core

Hour	Task
H1-H2	Decoupling
H3	HR kinetics
H4	Drift + blow-up
H5	Seeded noise
H6	Step-response test
H7	Commit

Day 6: simulator packets and faults

Hour	Task
H1	Crank events
H2	Packet scheduler
H3	TOML scenarios
H4	Faults + CTRL
H5	8 scenario files
H6	Loopback test
H7	Commit

Day 7: vc-core + .zwo

Hour	Task
H1	Ingest path
H2	Workout timeline
H3	SafetyGate
H4-H5	Arbiter
H6	<code>zwo.rs</code>
H7	Commit

Day 8: golden logs, lint, CLI, FFI

Hour	Task
H1	Golden log tests
H2	Phrase lint
H3	vc-cli
H4	vc-ffi
H5	Inspect generated bindings
H6	<code>build-ios.sh</code>
H7	<code>build-android.sh</code>

DoD: both artifacts build.

Day 9: Android peripheral

Hour	Task
H1	Manifest + permissions
H2	Foreground service
H3	<code>GattRig.kt</code>
H4	CCCD handling
H5	<code>Advertiser.kt</code>
H6	<code>RadioLoop.kt</code>
H7	Check from the Mac with btleplug

Day 10: iOS central

Hour	Task
H1	Link VcCore
H2	<code>BleCentral.swift</code>
H3	Ingest
H4	Tick + snapshot
H5	<code>RideView</code>
H6	Reconnect
H7	15-minute run

Day 11: audio, end-to-end, tag v0.1.0

Hour	Task
H1-H2	phrasegen + generate the phrase bank
H3	<code>AudioCuePlayer.swift</code>
H4	Wire up the callbacks
H5-H6	End-to-end run + fixes
H7	Tag v0.1.0

Day 12: clock sync

Hour	Task
H1	<code>sync.rs</code>
H2	Android PONG
H3	iOS PING
H4	STAMP correlation
H5	Latency histograms
H6	CSV export
H7	DEVLOG

Day 13: measure and optimize

Hour	Task
H1-H2	Measurement runs
H3	Latency report
H4	Fix the largest segment
H5	Re-measure
H6	LATENCY.md
H7	Commit

Day 14: faults and resilience

Hour	Task
H1	iOS debug sheet
H2-H3	Walk the resilience matrix on device
H4	Audio interruptions
H5	Adapter / permission screens
H6	Re-sign the iOS app
H7	DEVLOG

Day 15: hardening, tag v0.2.0

- Golden logs ×3, 30 min fuzz per target, allocation test, criterion, CI green.
- Tag v0.2.0.

Day 16: Intervals.icu

Hour	Task
H1	<code>icu.rs</code>
H2	<code>vc-cli icu pull</code> (Basic auth: username <code>API_KEY</code> , password = your key; athlete <code>0</code>)
H3	<code>IcuClient.swift</code>
H4	Confirm <code>resolve=true</code> behaviour [UNVERIFIED]
H5	Countdowns
H6-H7	Buffer

Day 17: soak test

- Two 60-minute runs with a fault injected every 10 min.
- DoD: S7 met.

Day 18: GC cross-check, CSV replay, `vc-cli live`, UI polish

Day 19: README, ENGINEERING_NOTES, LATENCY.md, licenses, dry run

Day 20: ship, tag v1.0.0

- Video, re-sign, 15-minute soak.
- Tag v1.0.0.
- Message Aki.

Fallbacks

- iOS can't discover the iQOO: use nRF Connect as the advertiser, or the Mac (btplug) as central.
- UniFFI on Kotlin is too painful: do the encoding in Kotlin for now.
- Earbud latency dominates the numbers: make the T3/T4 split the story.

Cut order (cut first → last): FIT replay → Waterworth model → criterion → Intervals.icu (keep .zwo import) → decoupling cue → GC cross-check.

Never cut: SafetyGate, Arbiter tests, golden logs, latency measurement, fuzzing.

Git workflow

- Trunk-based on `main`. Short branches such as `feat/gatt-cpm`, squash-merged through CI.
- Conventional commits, e.g. `feat(vc-gatt): decode CPM crank data`.
- Commit at least every 2 hours.
- Tags: v0.1.0 (day 11), v0.2.0 (day 15), v1.0.0 (day 20).

Deliverables

- **README:** why (Aki's rules), GIF, diagram, hardware-in-the-loop explanation, quick start, crate map, tests, latency summary, limitations, roadmap.
- **LATENCY.md template:** columns Segment | Route | n | p50 | p90 | p99 | max | sync ±. Rows: T1→T2, T1→T3, T0→T1, T0→T3, T0→T4 (earbuds), T0→T4 (speaker).
- **ENGINEERING_NOTES outline:** sans-IO, W'bal, Arbiter, clock-sync bound, GATT gotchas, iOS cache, Vivo, simulator provenance, next steps.
- **Video storyboard (3 min):**

Time	Shot
0:00	Hook
0:15	Architecture
0:45	Live <code>vo2_5x3</code> run with the W'bal cue
1:45	HR dropout
2:10	Redline pre-empting another cue
2:35	Latency numbers + CI
2:50	The ask

- **Message to Aki:** "Hi Aki — ahead of the trial I built `velocoach-rs`: a Rust core (`no_std` GATT codecs, `GoldenCheetah-parity` W'bal, a deterministic arbiter enforcing your verb-first/one-number/8-12-syllable rule and the 98%-HRmax safety bypass) running on an iPhone via `UniFFI`, fed by an Android phone posing as a power meter and HR strap over real BLE. Measured p99 [X] ms sensor-to-audio-submit; numbers and a 3-minute video: [link]. — Zakir"

3. "Explain it in the trial" cheat sheet

- **Hardware-in-the-loop over real radio:** exercises the real BLE failure surface at zero cost, while the simulator's ground truth makes correctness checkable.
- **Sans-IO core:** deterministic, replayable on the Mac, and the `no_std` crates are ready to port to an MCU.
- **Event-driven decisions:** cue decisions take microseconds, not up to a 250 ms tick.
- **W'bal:** `GoldenCheetah`'s differential model, recovery constant $W'/(CP - P)$, solved exactly for irregular sample spacing.
- **Safety outside the "AI":** deterministic, ignores cooldowns, and is the only thing allowed to talk over another cue.

- **The Arbiter reduces nagging:** priorities, cooldowns, an 8 s global gap, 2 s staleness, 25/35% hysteresis, structure reservation.
- **Pre-rendered TTS:** zero synthesis latency and phrases that can be linted.
- **Latency honesty:** $\pm \text{rtt_min}/2$ clock-sync bound, with T3 and T4 reported separately.
- **UniFFI 0.32:** zero-copy (`&[u8]`), one Mutex-guarded Object per platform, synchronous API.

Caveats

- I could not open the current GATT Specification Supplement; the layouts come from the SIG XML definitions plus consistent implementations. CP Feature bits ≥ 12 and the internal types of the CPM extreme fields are unverified.
- The simulator's physiology constants are tuned defaults, not cited values.
- These items are unverified, and each has a check scheduled in the plan:
 - NDK r30 install path
 - latest JNA version
 - Piper CLI flags and voice license
 - Intervals.icu (`resolve=true`) behaviour
 - Vivo battery-setting names
 - Background Modes on a free Apple team
 - the 160 ms A2DP figure
- UniFFI 0.32.x is recent (0.32.0 released 2026-06-30, 0.32.2 on 2026-09-22). Keep it pinned at =0.32.2.